Digital Image Processing Homework 1-6

General Guidelines

For **Homeworks 2–6**, every algorithm must be implemented twice in Python on the supplied images:

1. **From Scratch:** Use NumPy indexing, loops, and explicit math. Do not call high-level OpenCV, SciPy, and/or Skimage functions for the core operation.
2. **Library Standard:** Use the corresponding OpenCV or SciPy function.

For each task, report:

1. Visual comparison (input, scratch output, library output, absolute difference).
2. MSE between scratch and library outputs:

$$\text{MSE} = \frac{1}{MNC} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \sum_{c=0}^{C-1} \left(I_{\text{scratch}}(x, y, c) - I_{\text{lib}}(x, y, c) \right)^2 .$$

3. Execution time using time.perf_counter().
4. Saved outputs with specified filenames.

Homework 1: Image Basics and Pixel Operations

1. Load LenaColor512.bmp, display it, print height, width, channels, dtype, and min/max per channel. Save as HW1_Q1.bmp.
2. Draw a solid red vertical line at the integer center column $x = W//2$, full height. Save as HW1_Q2.bmp.
3. On the previous output, draw a blue line of thickness 2 from (100, 100) to (200, 200). Save as HW1_Q3.bmp.
4. On the previous output, set the rectangle from top-left (50, 50) to bottom-right (100, 100) inclusive to solid green by direct array assignment. Save as HW1_Q4.bmp. 5. Split the colour image into R, G, B channels and display each as grayscale. Convert to HSV, **CIELAB** (cv2.COLOR_BGR2LAB) and YCrCb, display all channels, and discuss their visual content. CIELAB is more perceptually uniform and is a modern, widely used alternative to HSV.

Homework 2: Intensity Transformations and Histogram Processing

1. **Image Negative:** $s = (L-1) - r$. Apply to grayscale and colour. Save scratch and OpenCV outputs.
2. **Gamma Transform:** $s = c\, r^{\gamma}$ with $r \in [0, 1]$, output rescaled to $[0, 255]$.

   Test $\gamma = 0.4, 0.67, 1.5, 2.5$.

   Display side-by-side.
3. **Log Transform:** $s = c \log(1 + r)$. Also implement piecewise-linear contrast stretching using control points (r_1, s_1) and (r_2, s_2).
4. **Histogram Equalization:** Compute histogram $p_r(r_k) = n_k/(MN)$, CDF $T(r_k) = (L-1) \sum_{j=0}^{k} p_r(r_j)$, and round mapped intensities. Compare with cv2.equalizeHist(). Plot histograms.

5. **Histogram Matching:** Match LenaGrey256.bmp to a specified target distribution. Sug gested references: a synthetic Gaussian target with $\mu = 128$, $\sigma = 40$, or a natural reference image such as LenaColor512.bmp converted to grayscale. Histogram matching does not require the reference to have the same spatial dimensions.

Homework 3: Resolution, Interpolation, and Geometric Transformations

1. **Spatial Resolution:** Downsample LenaColor512.bmp to $2^k \times 2^k$ for $k = 8, 7, 6, 5, 4$. Upsam ple back to 512 × 512, display grid, and analyze pixelation.
2. **Intensity Resolution:** Quantize to 2^k levels for $k = 8, 7, \ldots, 1$. Display results and identify false contours.
3. **Interpolation:** Crop a 64×64 ROI and upscale by 2× and 4× using nearest-neighbor, bilinear, bicubic, and Lanczos (or SciPy spline). Compare with cv2.resize using MSE and visual inspection.
4. **Geometric Transformations:** Use backward mapping and explicit boundary handling for: • Translation by (40, 40).
   • Scaling by 2.0 and 0.5.
   • Rotation by 45° about the image center with bilinear interpolation.

Homework 4: Spatial Domain Filtering

1. **Low-pass Filtering:**
   • Box filters: 3 × 3, 5 × 5, 9 × 9.
   • Gaussian filters: 5 × 5 and 11 × 11, $\sigma = 1.0, 2.5$. Normalize each discrete kernel to sum to 1.
   • Median filtering: Add 10% salt-and-pepper noise to LenaGrey256.bmp, then apply 3×3 mean and 3×3 median filtering. Median is nonlinear and not implemented as ordinary convolution. Compare MSE and edge preservation.
2. **High-pass / Sharpening:**
   • Laplacian kernels:

$$K_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}, \quad K_2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

   Sharpen via $g = f - \nabla^2 f$ or $g = f + \nabla^2 f$ depending on the center sign. • Unsharp masking and high-boost:

$$g_{\text{mask}} = f - f_{\text{smooth}}, \quad g = f + k\, g_{\text{mask}},$$

   with $k = 1.0$ (unsharp) and $k = 4.5$ (high-boost).
   • Sobel gradients:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

q
$g^2_x + g^2_y$ and phase $\alpha = \arctan(g_y/g_x)$

Compute and display images of magnitude

$M =$ (side by side).

2

3. **Combined Processing:** Use LenaGrey256.bmp, with intensities scaled to [100, 150] to re duce contrast artificially.

(a) $g_a = f - \nabla^2 f$ (Laplacian sharpening).

q

(b) $g_b =$ magnitude).

$g^2_x + g^2_y$(Sobel

(c) $g_c =$ box-filter g_b with 5 × 5 kernel.

(d) $g_d = g_a \cdot g_c$.

(e) $g_e = f + g_d$.

(f) $g_f = c\, g^{0.5}$

$_e$(gamma).

Display all intermediate outputs and explain each stage.

Homework 5: Frequency Domain Analysis

Use the standard DFT convention compatible with np.fft.fft2:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})},$$

with inverse

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v)\, e^{+j2\pi(\frac{ux}{M} + \frac{vy}{N})}.$$

1. **DFT vs FFT:** Implement 1D DFT $O(N^2)$ and radix-2 FFT $O(N \log N)$, then extend to 2D by row-column separability. Benchmark against np.fft.fft2 for $N \in \{16, 32, 64, 128, 256\}$. 2. **Spectrum and Phase:** Center the spectrum using fftshift or by multiplying by $(-1)^{x+y}$ before and after the transform. Display $\log(1 + |F|)$.

• Phase-only reconstruction: set $|F| = 1$, keep phase, inverse transform. • Magnitude-only reconstruction: set phase to zero, keep $|F|$, inverse transform. Compare the two reconstructions.

3. **Low-pass Filtering:** With $D(u, v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$, implement ideal, Butterworth ($n = 2$), and Gaussian low-pass filters. Apply to LenaGrey256.bmp for $D_0 = 10, 30, 60$. Reconstruct and analyze ringing/Gibbs artifacts.
4. **High-pass Filtering:** Use $H_{HP} = 1 - H_{LP}$ with $D_0 = 30$. Apply high-frequency emphasis

$$H_{\text{HFE}} = a + bH_{\text{HP}}, \quad a = 0.5, \; b = 2.0.$$

Post-process with histogram equalization and evaluate contrast.

Homework 6: Frequency Applications and Advanced Transforms

1. **Periodic Noise and Notch Filtering:**
   - Add sinusoidal noise $n(x, y) = 50 \sin(2\pi \cdot 32x/M + 2\pi \cdot 32y/N)$.
   - Display the centered log-magnitude spectrum showing the two impulse spikes.
   - Design a Butterworth notch filter centered at $(32, 32)$ and $(-32, -32)$:

   $$H_{\text{NRF}} = \frac{1}{1 + \left[\frac{D_0^2}{D_1(u,v)D_2(u,v)}\right]^n},$$

   with

   $$D_1 = \sqrt{(u - M/2 - 32)^2 + (v - N/2 - 32)^2}, \quad D_2 = \sqrt{(u - M/2 + 32)^2 + (v - N/2 + 32)^2}. \quad 3$$

   Apply filter, inverse transform, and compare with the uncorrupted image using MSE and SSIM.
2. **Homomorphic Filtering:** Model $f = i \cdot r$. Normalize f to $[0, 1]$ or use $\ln(f + 1)$, then:

   $$z = \ln(f + 1), \quad Z = \mathcal{F}\{z\},$$

   $$H_{\text{homo}} = (\gamma_H - \gamma_L)\left[1 - \exp\left(-c\frac{D^2}{D_0^2}\right)\right] + \gamma_L,$$

   with $\gamma_L = 0.25$, $\gamma_H = 2.0$, $c = 1.0$, $D_0 = 30$. Then

   $$s = \mathcal{F}^{-1}\{H_{\text{homo}}Z\}, \quad g = \exp(\text{Re}(s)) - 1.$$

   Clip the final output. Explain the roles of γ_L and γ_H.
3. **DCT and Energy Compaction:**
   - Implement 2D DCT-II and IDCT-II from scratch:

   $$C(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\frac{(2x + 1)u\pi}{2N} \cos\frac{(2y + 1)v\pi}{2N},$$

   where $\alpha(0) = \sqrt{1/N}$, $\alpha(u) = \sqrt{2/N}$ for $u > 0$. Match the scaling convention of cv2.dct.
   - Extract an 8×8 block and compute its FFT and DCT.
   - Retain only the top-left 3×3 coefficients. For the FFT, first center the spectrum and preserve conjugate symmetry; for the DCT, zero all coefficients outside the 3×3 top left block.
   - Reconstruct both blocks, compute MSE, and explain why DCT is preferred for JPEG like compression.